CLI Command Reference

AI-Driven Compiler Optimization System

Complete reference for every command you can run against a C/C++ source file.

1 · Full 3-Agent Pipeline — run_pipeline.py

Runs **Analysis** → **Optimization** → **Verification** end-to-end.

```
python run_pipeline.py &lt;file&gt;
```

Options

Flag	Short	Effect
--verbose	-v	Show every step in DEBUG detail (LLM prompts, regex hits)
--quiet	-q	Show warnings and errors only
--no-llm		Skip Ollama — use rule-based analysis only (faster, offline)
--show-messages	-m	Print live inter-agent messages on stdout
--save-messages FILE		Save inter-agent messages as JSON to FILE

Examples

```
# Default (INFO level logging)
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Verbose — see every LLM prompt and regex hit
python run_pipeline.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --verbose

# Quiet — only errors/warnings
python run_pipeline.py MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --quiet

# Skip LLM (much faster, no Ollama needed)
python run_pipeline.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --no-llm

# Show inter-agent messages live
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --show-messages

# Save messages to a JSON file
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --save-messages logs/messages.json

# Combine flags
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --no-llm --verbose
```

2 · Single-Agent Commands — run_agent.py

Run exactly **one** agent without the others.

```
python run_agent.py &lt;command&gt; &lt;file&gt; [options]
```

Sub-commands

Sub-command	Agents run	Output
analyze	Analysis only	Findings list with severity, line numbers
optimize	Analysis → Optimization	Diff + saved OPT_<file>
verify	Verification only	4-layer verification report

2a · Analyze only

```
python run_agent.py analyze &lt;file&gt; [--no-llm] [-v|-q]

# Basic analysis
python run_agent.py analyze MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Rule-based only, no Ollama
python run_agent.py analyze MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm

# Verbose output
python run_agent.py analyze MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --verbose

# Quiet (errors only)
python run_agent.py analyze MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --quiet
```

2b · Optimize only

Runs a quick Analysis internally first (to collect context), then applies optimizations. No verification pass.

```
python run_agent.py optimize &lt;file&gt; [--no-llm] [-v|-q]

# Optimize with LLM
python run_agent.py optimize MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Optimize offline (rule-based fixes only)
python run_agent.py optimize MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm

# Verbose
python run_agent.py optimize MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp --verbose
```

Optimized file is saved to:

```
MicroBenchmarks/Generated_optimisation/OPT_&lt;original_filename&gt;.cpp
```

2c · Verify only

Runs the 4-layer verification (Differential Testing → Z3 SMT → Performance Benchmark → LLM Reasoning). Needs an already-optimized file.

```
python run_agent.py verify &lt;file&gt; [--optimized &lt;opt_file&gt;] [-v|-q]

# Auto-detect optimized file from Generated_optimisation/
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Point to a specific optimized file
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp \
    --optimized MicroBenchmarks/Generated_optimisation/OPT_TC01_uninit_arithmetic.cpp

# Verbose verification (see each layer's detail)
python run_agent.py verify MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp \
    --optimized MicroBenchmarks/Generated_optimisation/OPT_TC01_uninit_arithmetic.cpp \
    --verbose
```

Auto-detect rule: if --optimized is omitted, looks for MicroBenchmarks/Generated_optimisation/OPT_.cpp

---

3 · Optimizer-Only Shortcut — run_optimizer.py

Focused entry point: **Analysis** → **Optimization**, no verification. Shorter output than run_agent.py optimize, supports custom output path.

```
python run_optimizer.py &lt;file&gt; [--no-llm] [--out &lt;path&gt;] [-v|-q]
```

Options

Flag	Effect
`--no-llm`	Rule-based fixes only
`--out <path>`	Custom path for the optimized file
`--verbose / -v`	DEBUG logging
`--quiet / -q`	WARNING-only logging

Examples

```
# Basic usage
python run_optimizer.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Rule-based only (no Ollama)
python run_optimizer.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp --no-llm

# Save to custom path
python run_optimizer.py MicroBenchmarks/Testcases/TC07_div_by_zero_var.cpp \
    --out MicroBenchmarks/Generated_optimisation/my_fixed.cpp

# Quiet with custom output
python run_optimizer.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp \
    --no-llm --out results/TC05_fixed.cpp --quiet
```

---

4 · Message Watcher — watch_agents.py

Like run_pipeline.py but streams **every inter-agent message** to the terminal in real time, with a final summary table.

```
python watch_agents.py &lt;file&gt; [--full] [--no-llm]
```

Options

Flag	Effect
`--full`	Show complete payloads including full source code and diffs
`--no-llm`	Skip Ollama

Examples

```
# Watch all messages (payloads truncated by default)
python watch_agents.py MicroBenchmarks/Testcases/TC11_buffer_overflow_loop.cpp

# Show full payloads (including source code)
python watch_agents.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp --full

# Rule-based mode
python watch_agents.py MicroBenchmarks/Testcases/TC05_uninit_struct.cpp --no-llm
```

---

5 · Command Comparison Table

Goal	Command
Full audit (all 3 agents + verify)	python run_pipeline.py <file>
Full audit + see agent messages	python run_pipeline.py <file> --show-messages
Full audit + save message log	python run_pipeline.py <file> --save-messages logs/msgs.json
Stream all messages live	python watch_agents.py <file>
Just detect bugs/patterns	python run_agent.py analyze <file>
Fix code, skip verification	python run_agent.py optimize <file>
Verify an existing optimized file	python run_agent.py verify <file> --optimized <opt>
Quick optimize, focused output	python run_optimizer.py <file>
Optimize offline (no Ollama)	python run_optimizer.py <file> --no-llm

---

6 · LLM Modes

All commands support **--no-llm** to skip Ollama entirely and use rule-based logic only.

LLM Mode	When to use	Speed
(default) — LLM + rules	Ollama is running (ollama serve)	Slower, richer output
--no-llm — rules only	No Ollama / quick offline scan	Fast

To start Ollama: `ollama serve` (in a separate terminal).

Model used: **Qwen 2.5 Coder** (pulled with `ollama pull qwen2.5-coder:7b`).

---

7 · Verbosity Levels

All commands support the same logging flags:

Flag	Level	Shows
`-v / --verbose`	`DEBUG`	`LLM prompts, regex hits, every internal step`
`(default)`	`INFO`	`One line per pipeline step — recommended`
`-q / --quiet`	`WARNING`	`Only warnings and errors`

Log file (always DEBUG, regardless of console setting):

```
logs/pipeline.log
```

---

8 · File Path Notes

Paths can be **relative** (from e:\SEM 4\CD_Refined\) or **absolute**.

Test cases are in `MicroBenchmarks/Testcases/` — filenames follow the pattern `TC<nn>_<description>.cpp`.

Optimized outputs are saved to `MicroBenchmarks/Generated_optimisation/OPT_<original>.cpp` unless overridden with `--out`.

Message logs (if using `--save-messages`) default to `logs/messages.json`.

```
# Relative path (most common)
python run_pipeline.py MicroBenchmarks/Testcases/TC01_uninit_arithmetic.cpp

# Absolute path
python run_pipeline.py "E:\SEM 4\CD_Refined\MicroBenchmarks\Testcases\TC01_uninit_arithmetic.cpp"
```